Asynchronous Steady-State ABC-SMC for Heterogeneous Simulator Workloads

First Author[1*] and Second Author[1]

[1*]Department, Organization, City, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;
Contributing authors: second.author@example.com;

Abstract

We study an asynchronous steady-state variant of approximate Bayesian computation with sequential Monte Carlo for simulator-based inference on heterogeneous high-performance computing workloads. The paper targets three questions: whether the method preserves posterior quality relative to established ABC baselines, whether asynchronous execution improves resource utilization under runtime heterogeneity, and whether the approach integrates cleanly into a practical distributed inference stack. This manuscript is currently a structured draft. It fixes the paper narrative, section flow, equations, planned experiments, and placeholder figures and tables while the experimental code is being completed.

Keywords: approximate Bayesian computation, sequential Monte Carlo, asynchronous algorithms, high-performance computing, likelihood-free inference

1 Introduction

Approximate Bayesian computation (ABC) is a standard tool for simulator-based inference when the likelihood is unavailable or too expensive to evaluate directly [1]. Sequential ABC methods improve efficiency by evolving proposal distributions and tolerance thresholds across a sequence of intermediate targets [2–4]. In practice, however, many ABC-SMC implementations remain organized around synchronous populations, which introduces generation barriers and idle compute resources when simulation runtimes vary strongly across parameter settings.

This paper focuses on a steady-state asynchronous alternative designed for heterogeneous high-performance computing workloads and implemented within Propulate. The intended contributions are:

1. a history-reconstructed asynchronous steady-state ABC-SMC algorithm compatible with Propulate's stateless propagator interface;
2. a practical formulation of monotone tolerance adaptation and rolling archive reconstruction from evaluated history;
3. an empirical evaluation against pyABC and other ABC baselines; and
4. an analysis of runtime heterogeneity, utilization, and scaling behavior on distributed systems.

The remainder of the paper first summarizes the relevant ABC background, then develops the asynchronous method, describes the implementation, and finally lays out the experimental program that will populate the final results section.

2 Background and Related Work

2.1 Approximate Bayesian computation

ABC replaces exact likelihood evaluation with simulation and discrepancy-based acceptance. For observed data y and parameter θ, the ABC target at tolerance ϵ can be written as

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta) L_\epsilon(\theta), \tag{1}$$

where $\pi(\theta)$ is the prior and $L_\epsilon(\theta)$ denotes the likelihood approximation induced by the discrepancy threshold. The final paper should expand this section with the benchmark-specific discrepancy definitions, summary statistics, and tolerance schedules used in the experiments.

2.2 Sequential ABC algorithms

ABC rejection provides the simplest baseline but scales poorly in low-tolerance regimes. ABC-SMC and ABC-PMC address this by moving through a sequence of intermediate targets with adaptive proposals and importance weights [2–4]. A common proposal form is the mixture

$$q_r(\theta) = \sum_{j=1}^{N_r} W_j^{(r)} K(\theta \mid \theta_j^{(r)}), \tag{2}$$

where K is typically a Gaussian perturbation kernel around accepted particles from the previous stage.

The main limitation for distributed execution is that these methods usually advance in generations: the proposal distribution and tolerance update only after an entire population has been evaluated. On heterogeneous workloads this creates a synchronization bottleneck.

2.3 Parallel and distributed ABC

Existing software such as pyABC demonstrates that ABC can be distributed effectively across multi-core and cluster environments [5, 6]. The key distinction for this paper

is not whether simulation is parallelized, but whether the inference algorithm itself requires synchronous population boundaries. That distinction motivates the event-driven formulation introduced next.

3 Method

3.1 Design constraints

The implementation target is Propulate, whose propagator interface exposes a single stateless call of the form `__call__(inds: List[Individual]) -> Individual`. The propagator receives the full evaluated history and must produce the next candidate without relying on persistent archive state or asynchronous callbacks. This leads directly to a history-based reconstruction of all algorithmic state.

Let $\mathcal{H}_n = \{x_1, x_2, \ldots, x_n\}$ denote the evaluated history after n completed simulations. The algorithm reconstructs

$$\epsilon_n = T(\mathcal{H}_n), \tag{3}$$

$$A_n = A(\mathcal{H}_n, \epsilon_n), \tag{4}$$

$$q_n(\theta) = Q(\theta \mid A_n), \tag{5}$$

where T is the tolerance scheduler, A is the archive selection rule, and Q constructs the proposal distribution from the reconstructed archive.

3.2 Tolerance reconstruction and monotonicity

Each proposed individual stores the effective tolerance active at proposal time. Given stored tolerances τ_i, the effective tolerance reconstructed from history is

$$\epsilon_{\text{hist}} = \min_{i:\tau_i \neq \varnothing} \tau_i, \tag{6}$$

with fallback to the initial tolerance when the history is empty. The scheduler proposes an updated tolerance ϵ_{sched}, and the final tolerance is

$$\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}}). \tag{7}$$

This gives a monotone decrease guarantee without mutable propagator state.

3.3 Archive reconstruction

The active archive is reconstructed by filtering the evaluated history according to the current tolerance:

$$A_n = \{\theta_i \in \mathcal{H}_n : \rho_i < \epsilon_n\}, \tag{8}$$

followed by truncation to the best k particles by discrepancy. This archive approximates the current ABC target and supplies the particles used for proposal construction.

3.4 Proposal and weighting

With archive particles $\{\theta_j\}_{j=1}^k$ and normalized weights $\{W_j^{(n)}\}_{j=1}^k$, the steady-state proposal is

$$q_n(\theta) = \sum_{j=1}^{k} W_j^{(n)} K(\theta \mid \theta_j), \tag{9}$$

using a Gaussian perturbation kernel with covariance estimated from the current archive. A newly proposed particle receives importance weight

$$w^\star = \frac{\pi(\theta^\star)}{q_n(\theta^\star)}. \tag{10}$$

The final manuscript should discuss the known approximation introduced by asynchronous execution: the proposal used to generate a particle is fixed at proposal time, while the archive may evolve before the simulation result returns.

3.5 Steady-state update loop

Algorithm 1 History-reconstructed asynchronous steady-state ABC-SMC

Require: Evaluated history $\mathcal{H}_n$, archive size k, initial tolerance ϵ_0

```
Reconstruct ε_hist from stored particle tolerances
Compute scheduler proposal ε_sched
Set ε_n ← min(ε_hist, ε_sched)
Reconstruct archive A_n from particles with ρ < ε_n
if |A_n| < k then
    Sample from the prior
else
    Build mixture proposal q_n(θ) from A_n
    Sample a parent and perturb to obtain θ*
    Clip to parameter bounds and stamp θ* with ε_n
    Compute the importance weight associated with the proposal at creation time
end if
Return the candidate for external simulation and discrepancy evaluation
```

3.6 Relation to classical ABC-SMC

Table 1 captures the paper's central algorithmic claim. The final version of this subsection should also make explicit which properties remain aligned with standard ABC-PMC theory and which aspects are practical approximations introduced for asynchronous HPC execution.

Table 1 High-level comparison between classical ABC-SMC and the proposed steady-state variant.

Property	Classical ABC-SMC	Steady-state async ABC
Update style	generation-based	event-driven
Synchronization	required per population	none
State storage	explicit population state	reconstructed from history
Tolerance changes	generation boundaries	online, monotone
Best fit for workloads	homogeneous runtimes	heterogeneous runtimes

4 Implementation in Propulate

This section will document the concrete implementation decisions in the codebase. The planning notes already fix the main structure:

- stateless tolerance schedulers implemented as pure functions of history;
- archive selection from the top-k accepted particles under the reconstructed tolerance;
- proposal covariance regularization for numerical stability; and
- integration with Propulate's MPI-oriented execution model.

The final write-up should include one concise code-level schematic, one paragraph on the propagator contract, and one paragraph on how metadata such as tolerance, weight, and generation are stored in individuals. It should also specify how pyABC baselines are configured to make the comparison fair.

5 Experimental Design

5.1 Research questions

The experiments are organized around three top-level questions:

1. Does the asynchronous steady-state method recover posteriors comparable to established ABC baselines?
2. Does asynchronous execution improve wall-clock efficiency and utilization under runtime heterogeneity?
3. Does the method remain practical on realistic simulator-driven benchmarks?

5.2 Benchmarks

The benchmark suite spans increasing levels of realism and computational heterogeneity. The Gaussian model anchors correctness against a tractable reference posterior, while the Cellular Potts model is intended to demonstrate the HPC advantages of barrier-free execution.

5.3 Baselines

The baseline set currently planned in the resources consists of:

Table 2 Planned benchmark suite for the paper.

Benchmark	Role in paper	Planned outputs
Gaussian mean inference	sanity check with analytic posterior	posterior overlays, error curves, calibration checks
g-and-k distribution	standard likelihood-free benchmark	posterior error, Wasserstein distance, convergence comparison
Lotka–Volterra system	established simulator-based ABC model	posterior recovery, simulation budget comparison
Cellular Potts model	realistic heterogeneous simulator workload	wall-clock, throughput, utilization, scaling results

- rejection ABC for simple low-dimensional settings;
- a classical ABC-SMC or ABC-PMC style baseline where practical; and
- distributed ABC-SMC via pyABC as the main external comparison.

5.4 Evaluation metrics

The paper separates statistical and computational evaluation.

Statistical accuracy.

Primary metrics will include posterior mean error, Wasserstein distance between posterior samples, and credible interval coverage. A benchmark-specific summary table should collect these values across methods and random seeds.

Computational performance.

Primary metrics will include wall-clock time to a fixed error target, simulation throughput, worker utilization, and idle fraction. Scaling experiments will measure speedup and efficiency as the worker count increases.

5.5 Runtime heterogeneity, scaling, and sensitivity studies

The planning notes specify three additional experiment families that should each receive a dedicated subsection in the final paper:

- synthetic runtime heterogeneity experiments, for example via log-normal runtime perturbations;
- strong-scaling experiments across worker counts such as 1, 8, 32, 128, and 256; and
- sensitivity and ablation studies covering archive size, tolerance schedules, covariance adaptation, and archive truncation rules.

Fig. 1 Placeholder for posterior comparison figures on the Gaussian and g-and-k benchmarks. Planned content: posterior overlays, calibration summaries, and error-versus-budget plots.

Fig. 2 Placeholder for runtime heterogeneity results. Planned content: worker utilization, throughput, and time-to-target comparisons for synchronous and asynchronous methods.

6 Results

This section is intentionally structured before the code and figures are complete. The final results should follow the same order as the research questions and use stable subsection titles so plots and tables can be dropped in incrementally.

6.1 Posterior quality on synthetic benchmarks

[TODO: Insert posterior comparison plots and quantitative accuracy table once the benchmark runners and aggregation scripts are complete.]

Fig. 3 Placeholder for scaling results. Planned content: parallel speedup and efficiency versus number of workers.

6.2 Performance under runtime heterogeneity

[TODO: Insert synthetic heterogeneity experiment showing asynchronous utilization benefits under parameter-dependent runtimes.]

6.3 Scaling on distributed resources

[TODO: Insert strong-scaling plots and the corresponding processed metrics table.]

6.4 Realistic simulator benchmark

[TODO: Summarize the Cellular Potts benchmark here, including both posterior behavior and HPC performance characteristics.]

7 Discussion

The discussion should interpret the trade-off between exact sequential ABC semantics and practical asynchronous execution. At minimum, the final version should address:

- when the proposal-weight staleness approximation is negligible in practice;
- which workload patterns most strongly favor asynchronous execution;
- how much statistical deviation from synchronous ABC-SMC is empirically acceptable; and
- what aspects of the approach are specific to Propulate versus portable to other distributed inference frameworks.

8 Conclusion

This paper aims to position asynchronous steady-state ABC-SMC as a practical compromise between the statistical structure of sequential ABC and the systems realities of heterogeneous distributed simulation. The final conclusion should return to the three

core claims of the paper: posterior validity, computational advantage, and practical usability.

Acknowledgements. [TODO: Add funding, compute allocation, and collaboration acknowledgements.]

Declarations

Funding
[TODO: Add funding statement.]

Conflict of interest/Competing interests
The authors declare no competing interests.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Data availability
[TODO: State where raw and processed experiment outputs will be archived.]

Materials availability
Not applicable.

Code availability
[TODO: Point to the repository and experiment reproduction entry point.]

Author contribution
[TODO: Add author contribution statement.]

Appendix A Implementation Details

This appendix can absorb implementation specifics that would otherwise clutter the main method section. Likely contents include tolerance scheduler definitions, covariance regularization details, and any benchmark-specific simulation notes that are useful for reproducibility.

Appendix B Additional Experimental Protocol

This appendix can collect resolved configuration summaries, seed handling, cluster setup, and any benchmark-specific details needed to reproduce the figures.

References

[1] Beaumont, M.A.: Approximate bayesian computation. Annual Review of Statistics and Its Application **6**, 379–403 (2019) https://doi.org/10.1146/annurev-statistics-030718-105212

[2] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate bayesian computation scheme for parameter inference and model selection in

dynamical systems. Journal of the Royal Society Interface **6**(31), 187–202 (2009) https://doi.org/10.1098/rsif.2008.0172

[3] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate bayesian computation. Biometrika **96**(4), 983–990 (2009) https://doi.org/10.1093/biomet/asp052

[4] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential monte carlo method for approximate bayesian computation. Statistics and Computing **22**(5), 1009–1020 (2012) https://doi.org/10.1007/s11222-011-9271-y

[5] Klinger, E., Rickert, D., Hasenauer, J.: pyabc: distributed, likelihood-free inference. Bioinformatics **34**(20), 3591–3593 (2018) https://doi.org/10.1093/bioinformatics/bty361

[6] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyabc 0.12.3: high-performance distributed, likelihood-free inference. Bioinformatics **38**(13), 3354–3355 (2022) https://doi.org/10.1093/bioinformatics/btac321